

Material de Lectura Extendido

Unidad 3: Machine Learning

Departamento de Educación a Distancia

Analítica de Big Data

Material de Lectura - Unidad 3: Machine Learning en Big Data con R

Año 2025

Unidad 3: Machine Learning en Big Data con R

El Machine Learning (ML) es esencial para extraer insights de grandes volúmenes de datos en Big Data. Esta unidad introduce conceptos clave de ML, enfocándose en modelos predictivos como K-Nearest Neighbors (KNN), Naive Bayes y Regresión Logística. Exploraremos explicaciones teóricas, ventajas/desventajas, ejemplos en R, diagramas y aplicaciones reales para un aprendizaje completo y práctico.

3.1 Introducción al Machine Learning en Big Data

ML permite a las máquinas aprender patrones de datos sin programación explícita. En Big Data, maneja volúmenes masivos con algoritmos escalables. Tipos: Supervisado (etiquetado), No Supervisado (patrones), Reforzamiento (recompensas). Enfoque en supervisado para clasificación/regresión.

Tabla 1: Tipos de Machine Learning

Tipo | Descripción | Ejemplos | Uso en Big Data

Supervisado | Aprende de datos etiquetados para predecir. | Clasificación, Regresión. | Predicción de churn en telecomunicaciones.

No Supervisado | Encuentra patrones sin etiquetas. | Clustering, Reducción dimensional. | Segmentación de clientes en e-commerce.

Reforzamiento | Aprende por ensayo-error con recompensas. | Q-Learning, Deep RL. | Optimización de rutas en logística.

Semi-Supervisado | Mezcla etiquetados y no etiquetados. | Autoencoders. | Clasificación de imágenes con datos limitados.

Fuente: Adaptado de GeeksforGeeks y DataCamp tutorials.

Error al insertar figura: 403 Client Error: Forbidden for url:

<https://www.researchgate.net/publication/342062164/figure/fig1/AS:900227347886082@1592604879453/Machine-Learning>

3.2 Creación de Modelos Predictivos

En ML supervisado, se divide datos en entrenamiento/test, se entrena el modelo y se evalúa (accuracy, precision, recall, F1-score). En R, paquetes como caret facilitan este flujo.

Ejemplo en R: Flujo Básico de ML con caret

R Code:

```
library(caret)# Cargar datos (ej. iris)data(iris)# Dividir en train/testset.seed(123)trainIndex <-
createDataPartition(iris$Species, p=0.8, list=FALSE)train <- iris[trainIndex,]test <- iris[-trainIndex,]# Entrenar modelo (ej.
KNN)model <- train(Species ~ ., data=train, method="knn")# Predecir y evaluarpredictions <- predict(model,
test)confusionMatrix(predictions, test$Species)
```

Este ejemplo usa caret para partición, entrenamiento y evaluación, común en Big Data para escalabilidad.

3.3 K-Nearest Neighbors (KNN)

KNN es un algoritmo no paramétrico de aprendizaje basado en instancias. Clasifica un punto nuevo basado en la mayoría de votos de sus K vecinos más cercanos, usando distancias como Euclídea. Ideal



para datasets pequeños a medianos; en Big Data, usar aproximaciones como KD-trees.

Fórmula de Distancia Euclídea: $\sqrt{\sum (p_i - q_i)^2}$

Tabla 2: Ventajas y Desventajas de KNN

Aspecto | Ventajas | Desventajas

Simplicidad | Fácil de entender e implementar. | Sensible a ruido y outliers.

No paramétrico | No asume distribución de datos. | Alto costo computacional en predicción.

Flexibilidad | Bueno para clasificación multi-clase. | Requiere elección óptima de K.

Escalabilidad | Útil en datasets no lineales. | No escala bien con Big Data sin optimizaciones.

Fuente: Adaptado de DataCamp KNN Tutorial.

Ejemplo en R: Implementación de KNN con class

R Code:

```
library(class)library(caret)# Datos irisdata(iris)# Normalizar (opcional)normalize <- function(x) { return ((x - min(x)) /
(max(x) - min(x))) }iris_norm <- as.data.frame(lapply(iris[1:4], normalize))# Train/test splitset.seed(123)trainIndex <-
createDataPartition(iris$Species, p=0.8, list=FALSE)train_data <- iris_norm[trainIndex,]train_labels <-
iris$Species[trainIndex]test_data <- iris_norm[-trainIndex,]test_labels <- iris$Species[-trainIndex]# KNN con K=3predictions
<- knn(train=train_data, test=test_data, cl=train_labels, k=3)# EvaluaciónconfusionMatrix(predictions, test_labels)
```

Este código normaliza features, aplica KNN y evalúa con matriz de confusión.

Figura 2: Diagrama de KNN

Fuente: GeeksforGeeks - KNN Algorithm.

3.4 Naive Bayes

Naive Bayes es un clasificador probabilístico basado en el Teorema de Bayes, asumiendo independencia entre features (naive). Eficiente para texto y Big Data, variantes: Gaussian (continuas), Multinomial (discretas), Bernoulli (binarias).

Teorema de Bayes: $P(C|X) = \frac{P(X|C) P(C)}{P(X)}$

Tabla 3: Ventajas y Desventajas de Naive Bayes

Aspecto | Ventajas | Desventajas

Eficiencia | Rápido en entrenamiento y predicción. | Asume independencia (puede no ser real).

Manejo de datos | Bueno con datos categóricos y texto. | Problemas con zero-frequency (Laplace smoothing).

Escalabilidad | Escala bien con Big Data. | Menos preciso en datasets complejos.

Simplicidad | Fácil interpretación probabilística. | No maneja bien interacciones entre features.

Fuente: Adaptado de Edureka Naive Bayes Tutorial.

Ejemplo en R: Naive Bayes con e1071

R Code:

```
library(e1071)library(caret)# Datos iris (para clasificación)data(iris)# Train/test splitset.seed(123)trainIndex <-
createDataPartition(iris$Species, p=0.8, list=FALSE)train <- iris[trainIndex,]test <- iris[-trainIndex,]# Entrenar Naive
Bayesmodel <- naiveBayes(Species ~ ., data=train)# Predecirpredictions <- predict(model, test[,1:4])#
EvaluaciónconfusionMatrix(predictions, test$Species)
```

Ejemplo simple con iris; en Big Data, útil para spam filtering o sentiment analysis.

Figura 3: Diagrama de Naive Bayes

Fuente: Medium - Mastering Naive Bayes.

3.5 Regresión Logística

Regresión Logística modela probabilidades para clasificación binaria/multinomial usando función sigmoide. Extiende regresión lineal para outputs categóricos.

Función Logística: $p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$

Tabla 4: Ventajas y Desventajas de Regresión Logística

Aspecto | Ventajas | Desventajas

Interpretabilidad | Coeficientes fáciles de interpretar. | Asume linealidad en logit.

Eficiencia | Rápida, no requiere mucho cómputo. | Sensible a multicolinealidad.

Probabilidades | Output probabilístico directo. | No maneja bien no-linealidades complejas.

Escalabilidad | Buena para Big Data con regularización. | Requiere datos balanceados.

Fuente: Adaptado de Statology Logistic Regression Tutorial.

Ejemplo en R: Regresión Logística Binaria con glm

R Code:

```
# Datos de ejemplo (titanic simplificado)
data <- read.csv("titanic.csv") # Asumir columnas: Survived, Pclass, Sex, Age, etc.
# Convertir factores
data$Survived <- factor(data$Survived)
data$Sex <- factor(data$Sex)
# Train/test split
set.seed(123)
trainIndex <- createDataPartition(data$Survived, p=0.8, list=FALSE)
train <- data[trainIndex,]
test <- data[-trainIndex,]
# Modelo logit
model <- glm(Survived ~ Pclass + Sex + Age, data=train, family="binomial")
# Predecir probabilidades
probabilities <- predict(model, test, type="response")
# Evaluación
confusionMatrix(factor(predictions), test$Survived)
```

Usando glm para clasificación binaria; en Big Data, aplicar en churn prediction o fraud detection.

Figura 4: Frontera de Decisión en Regresión Logística

Fuente: Medium - Logistic Regression and Decision Boundary.

3.6 Comparación de Algoritmos

Selección depende de datos: KNN para no-lineales, Naive Bayes para texto, Logística para interpretabilidad.

Tabla 5: Comparación de Algoritmos

Algoritmo | Tipo | Fortalezas | Debilidades | Uso en Big Data

KNN | Basado en instancias | Simple, no asume distribución. | Lento en predicción. | Clasificación local con aproximaciones.

Naive Bayes | Probabilístico | Rápido, maneja alta dimensionalidad. | Asume independencia. | Análisis de texto/spam en streams.

Regresión Logística | Lineal | Interpretabilidad, probabilidades. | Lineal en logit. | Predicciones binarias escalables.

Fuente: Elaboración propia basada en tutorials de DataCamp y GeeksforGeeks.

3.7 Aplicaciones Reales en Big Data

ML en Big Data: Recomendadores (Netflix - KNN), detección spam (Gmail - Naive Bayes), riesgo crediticio (bancos - Logística).

Tabla 6: Aplicaciones Reales

Algoritmo | Aplicación | Ejemplo | Beneficio

KNN | Sistemas de recomendación. | Spotify playlists. | Personalización basada en similitud.

Naive Bayes | Clasificación de texto. | Filtro spam en email. | Procesamiento rápido de volúmenes masivos.

Regresión Logística | Predicción binaria. | Churn en telecom. | Evaluación de probabilidades de retención.

Fuente: Adaptado de R-bloggers y Analytics Vidhya.

Conclusiones de la Unidad 3

ML transforma Big Data en insights accionables. KNN, Naive Bayes y Regresión Logística son bases sólidas; en R, paquetes como caret/e1071 facilitan implementación. Futuro: Integración con Spark para escalabilidad extrema.

Bibliografía

· DataCamp. (2023). K-Nearest Neighbors (KNN) Classification with R Tutorial.



<https://www.datacamp.com/tutorial/k-nearest-neighbors-knn-classification-with-r-tutorial>

· Edureka. (2019). A Step By Step Guide To Implement Naive Bayes In R.

<https://medium.com/edureka/naive-bayes-in-r-37ca73f3e85c>

· Statology. How to Perform Logistic Regression in R (Step-by-Step).

<https://www.statology.org/logistic-regression-in-r/>

· GeeksforGeeks. (2025). Introduction to Machine Learning in R.

<https://www.geeksforgeeks.org/r-machine-learning/introduction-to-machine-learning-in-r/>

· Additional: R-bloggers. (2021). Naive Bayes Classification in R.

<https://www.r-bloggers.com/2021/04/naive-bayes-classification-in-r/>

· Wickham, H. (2016). ggplot2: Elegant Graphics for Data Analysis. Springer.

· Lutz, M. (2013). Learning Python. O'Reilly Media (for general ML context).

